

LinkedHashMap

Predictable Identity & Insertion Flow

CODEHIVE

CODEHIVE

1. Beyond Standard Hashing

A **LinkedHashMap** is a hybrid data structure that combines the lightning-fast lookups of a **HashMap** with the sequential logic of a **LinkedList**.

The Core Advantage:

- **Insertion Order:** It remembers exactly which item was added first, second, and last.
- **Stability:** While a standard **HashMap**'s order is chaotic, **LinkedHashMap** is rock-solid and predictable.
- **Uniqueness:** It maintains the Map contract—keys must be strictly unique.

2. Creating the Map

To use this structure, import it from `java.util`. For professional applications, always declare using the `Map` interface to decouple your logic from the implementation.

```
import java.util.Map;
import java.util.LinkedHashMap;

// The Professional Way
Map<String, String> capitals = new LinkedHashMap<>();
```

3. Adding Entries

Items are added using the `put ( )` method. Since internal ordering is based on insertion, "England" will always be the first item encountered in a loop.

```
capitals.put("England", "London");  
capitals.put("India", "New Delhi");  
capitals.put("Austria", "Wien");  
capitals.put("Norway", "Oslo");  
capitals.put("USA", "Washington DC");
```

Notice: If you re-add "England" with a different value, the value is updated, but the entry usually retains its original position in the insertion sequence.

4. Access & Utility

Retrieving data is performed via the `get()` method. Additionally, the `size()` method provides the current entry count.

```
// Direct Access
String city = capitals.get("Norway"); // Oslo

// Size Check
int count = capitals.size(); // 5
```

5. Modifying the Set

Entries can be removed specifically by key or the map can be cleared entirely. Because of the internal doubly-linked list, pointers are updated during removal to maintain the sequence.

```
// Remove one entry
capitals.remove("USA");

// Wipe the map
capitals.clear();
```

6. Ordered Iteration

When looping through a LinkedHashMap, the order of elements follows the sequence in which they were originally added.

```
// Loop through Keys (Always predictable order)
for (String country : capitals.keySet()) {
    System.out.println(country);
}
```

7. Traversing Values

You can also loop through the values independently, or print the key-value pairs together for a complete data overview.

```
// Loop through Keys and Values
for (String country : capitals.keySet()) {
    System.out.println("Key: " + country +
        ", Value: " + capitals.get(country));
}
```


8. Local Type Inference

Modern Java (10+) allows the `var` keyword for cleaner declaration of local variables within a method scope.

```
// Syntax simplification  
var cityMap = new LinkedHashMap<String, String>();
```

9. HashMap vs LinkedHashMap

Understanding the performance trade-offs is critical for choosing the right tool.

Feature	HashMap	LinkedHashMap
Iteration Order	Unpredictable	Insertion Order
Performance	Highest ($O(1)$)	High (Slight overhead)
Memory Usage	Standard	Higher (Doubly-Linked)

© 2026 CodeHive Academy | Collections Series Final